Computers and Artificial Intelligence, Vol. 18, 1999, No. 6, 1—

A MORPHOLOGICAL APPROACH TO HOUGH TRANSFORM ON AN INSTRUCTION SYSTOLIC ARRAY

Bertil SCHMIDT
Manfred SCHIMMLER
Heiko SCHRÖDER

Institut für Datenverarbeitungsanlagen, TU Braunschweig
Hans-Sommer-Str. 66, 38106 Braunschweig, Germany
e-mail: bschmidt@ida.ing.tu-bs.de, masch@ida.ing.tu-bs.de,
asheiko@ntu.edu.sg

Abstract. Instruction systolic arrays have been developed in order to combine the speed and the simplicity of systolic arrays with the flexibility of MIMD parallel computer systems. Instruction systolic arrays are available as quadratic arrays of small RISC processors capable of performing integer and floating point arithmetic. In this paper a new algorithm for line detection is presented which applies the morphological approach to the well known Hough transform. The quality of its results is significantly higher than that of the classical Hough transform. Our algorithm has an AT-complexity of $O(N^3)$. This matches the one of the best known alternatives in the literature. It will be shown that the new algorithm is more efficient in practical applications. It has been tailored towards the capabilities of the instruction systolic array. This leads to a high speed implementation on Systola 1024, the first low cost parallel computer of this particular architecture on the market.

Keywords: Hough transform, mathematical morphology, low cost parallel computing, instruction systolic array

1 INTRODUCTION

Nowadays it is often argued that due to the ever increasing power of PCs and workstations parallel computing is losing its justification. We believe that there is some truth to this, e.g. cost efficiency does not allow for a massively parallel general purpose machine. In this paper we argue that there are niches where massively parallel special purpose computing does deliver the best cost efficiency. We believe

that there is a range of such niches and we show that many image processing applications on instruction systolic arrays belong into these niches [19].

Instruction systolic arrays (ISAs) provide a programmable high performance hardware for specific computation intensive applications [10, 11]. The commercially available ISA – Systola 1024 – is connected to a sequential host, thus operating like a coprocessor which solves only the computationally intensive tasks within a global application. The ISA model is a mesh connected processor grid, where the processors are controlled by three streams of control information: instructions, row selectors, and column selectors.

Hough transform (HT) is a standard technique for line detection in image processing. It works by transforming edge pixels in the image space into a parameter space and collecting in an accumlator array these pixels which satisfy the equation of a straight line. The HT continuously receives much attention because of its usefulness both as a tool in industrial applications and a step in building perceptual representations for computer vision tasks. Research on the HT method has progressed by improvements in the sequential version of the HT and in architectural support for its parallel implementation. Since straight lines are considered over the whole image, there are some global communication operations required for the transformation from the image space into the accumulator space. But the mesh does not allow global communication in a constant number of steps. Data movements from the border to opposite border requires $\Omega(n)$ steps. Several authors [2, 3, 4, 8, 14, 15, 18] use a more powerful computer model in order to overcome this disadvantage:

In [18] *wrap around* connections and broadcast facilities are introduced to speed up the mesh. Global buses by a *reconfigurable mesh* are used in [3, 8]. A SIMD hypercube instead a mesh allows for an efficient implementation as shown in [15]. Other implementations as in [2, 14] are tailored towards the individual communication patterns required for the HT.

One asymptotically optimal approach has been presented in [5]: All possible straight lines are traced simultaneously starting at one border of the mesh. Unfortunately, for practical implementations this algorithm shows major disadvantages: First: the processors have to perform complicated operations (e.g. sine and division); second: intersecting lines produce a collision of at most three items in one processor, introducing a delay of a constant factor of three; third: the algorithm consists of eight trace sequences. This introduces an additional delay of two in comparison to our algorithm presented here.

In this paper we present an asymptotically optimal solution that overcomes these problems. We take advantage of the ISA, a mesh with pipelined instruction execution. Due to efficient implementation of global communication on such an array processor no wrap around connections and no global buses are required. Furthermore, our algorithm uses only simple arithmetic operations like addition and logical operations like and, or, and not. Another major advantage of the ISA

architecture is that it not only allows the standard HT to be implemented but also the mathematical morphology version of it.

This paper is organised as follows. The concept of the ISA is explained in detail in the second section. The details of HT are given in Section 3. The classical HT creates artefacts if there are structures in the image like dashed lines or small line segments. Therefore, modifications of HT have been developed [6, 13]. In this paper an approach based on mathematical morphology is taken to find an efficient implementation of HT that reduces the number of artefacts in the transform space. This algorithm is explained in Section 4. Section 5 introduces the Systola 1024, a parallel computer with ISA architecture. Systola 1024 is available for a price in the range of industrial PCs [12]. We will use the syntax conventions of this architecture to explain the ideas behind the implementation of the morphological HT with an ISA in Section 6. Section 7 discusses the performance in comparison to implementations on a sequential architecture. The results of the original HT and those of the new algorithm are demonstrated in an example. The comparison of the efficiency of different HT solutions is provided in Section 8. In order to provide a fair comparison we analyse our algorithm in terms of area-time (AT)-complexity in Section 7.

2 PRINCIPLE OF THE ISA

The ISA is a quadratic array of identical processors, each connected to its four direct neighbours by data wires. The array is synchronised by a global clock. The processors are controlled by instructions, row selectors, and column selectors.

The instructions are input in the upper left corner of the processor array, and from there they move step by step in horizontal and vertical direction through the array. This guarantees that within each diagonal of the array the same instruction is active during each clock cycle. In clock cycle $k+1$ processor $(i+1, j)$ and $(i, j+1)$ execute the instruction that has been executed by processor (i, j) in clock cycle k.

The selectors also move systolically through the array: the row selectors horizontally from left to right, the column selectors vertically from top to bottom (Fig. 1). The selectors mask the execution of the instructions within the processors, i.e. an instruction is executed if and only if both selector bits, currently in that processor, are equal to one. This construct leads to a very flexible structure which creates the possibility of very efficient solutions for a large variety of applications, e.g. image processing, cryptography, and numeric.

Every processor has read and write access to its own memory. Besides that, it has a designated communication register (C-register) that can also be read by the four neighbour processors. Within each clock phase reading access is always performed before writing access. Thus, two adjacent processors can exchange data within a single clock cycle in which both processors overwrite the contents of their own C-register with the contents of the C-register of their neighbours. This convention

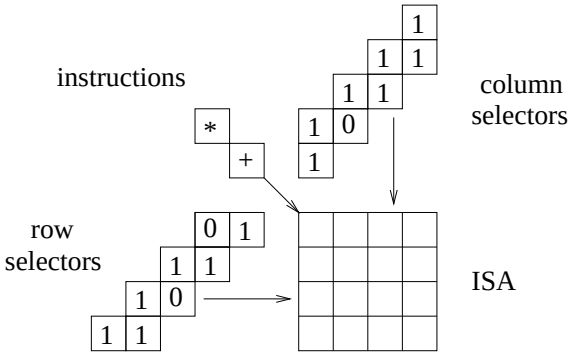

Fig. 1. Control flow in an ISA

avoids read/write conflicts and also creates the possibility to broadcast information across a whole row or column with one single instruction. This property can be exploited for an efficient calculation of row sums and row ringshifts which are the key operations in the parallel HT implementation described in Section 6:

Row sum. One important advantage of ISAs is the capability of performing aggregate functions within one (or a constant number of) instructions. Aggregate functions are operations where every processor needs information of all processors with smaller column index and the same row index (or vice versa). The simplest example is the computation of the row sums: Each processor computes the sum of the C-registers of its left neighbour and itself. Since the execution of this operation is pipelined along the row each processor accumulates the sum of all C-registers up to its own which is identical to the prefix sum.

Row ringshift. The contents of the C-registers can be ringshifted along the processor rows by two instructions. Every two horizontally adjacent processors exchange data (using one read left and one read right operation). Because of the instruction flow from west to east this implements a ringshift. Of course, a column ringshift can be executed in the same way.

The complexity of an algorithm on an ISA is the number of instructions necessary for its implementation. The ISA concept combines flexibility and speed. The possibility of implementing arbitrary MIMD array programs on the ISA proves its flexibility. Also arbitrary systolic arrays can be implemented on the ISA carrying over the area efficiency, speed, and low cost of systolic arrays to the ISA concept [20].

3 HOUGH TRANSFORM

The classical HT [6] is a very powerful robust technique to detect straight lines in binary images. It transforms the image into a parameter space by counting

pixels on straight lines. For every possible straight line in the image a counter is introduced which accumulates the number of pixels which satisfy the line equation $y = mx + c$. In the parameter space we represent a straight line by these two parameters: the slope m and the intercept (the intersection point with the y-axis) c. This representation is rather simple. The set of straight lines intersecting in one point of the original image is represented by a straight line in the m-c-space. The disadvantage of this representation is the fact that vertical lines (with infinite slope) cannot be represented. This problem can be eliminated by considering only slopes m with $-1 \leq m \leq 1$. By applying the HT to the original and the transposed of the input image the obtained discrete parameter domain estimates the wanted HT adequate.

The HT algorithm proceeds now in the following way: An accumulator array $B[m_k, c_l]$ is introduced to represent the parameter space. For every white pixel in the image and every slope m_k the corresponding value c_l is computed from the line equation and the counters B for all straight lines intersecting in this pixel are incremented by one. By that at the end of the execution the value of each counter represents the fraction of the corresponding straight line in the original image. The local maxima of the B array indicate the presence of lines of certain slopes and intercepts in the image. Thus, the problem of line detection in an image is replaced by an easier local peak detection in the accumulator.

The HT algorithm in the simplest form for a binary image $I(i, j)$ of size $N \times N$ and an accumulator array $B[m_k, c_l]$, where $k = 0, \ldots, M-1$, $m_k = \frac{k}{M}$, is shown in Fig. 2.

```
for i := 0 to N − 1 do
    for j := 0 to N − 1 do
        if I(i, j) = 1 then
            for k := 0 to M − 1 do increment B[m_k, round(j − i · k/M)]
```

Fig. 2. Standard HT

One disadvantage of HT is the high requirement in computing power such that online computations are impossible on existing sequential computers. The work is of the order N^2M. Therefore, a parallel implementation is useful and necessary for many applications. Another disadvantage is the fact that HT creates artefacts by counting structures in the image that finally turn out to be only very small fractions of straight lines (e.g. only one dot). In [1] a new method to address this problem is presented. Its basic idea and the new algorithm derived from it is presented in the next section.

4 MORPHOLOGICAL HOUGH TRANSFORM (MHT)

The idea of the MHT is to reduce artefacts in the accumulator array by combining the global character of the HT with a local morphological operation. Mathematical morphology is a technique well suited for image processing applications. Complex functions on binary images are decomposed into sequences of atomic operations of two different types: erosion and dilation. Both need a structural element. This is a small image segment, that is moved over the image such that a predefined reference point of the structural element is subsequently aligned with every pixel of the image.

The erosion matches the structural element in the image, i.e. if all pixels of the structural element are set in the image, the corresponding pixel in the erosion is set too. The dilation checks whether there is an intersection of the structural element with the image. A pixel of the dilation is set if and only if the structural element an the image have at least one pixel in common. Fig. 3 gives an example of erosion and dilation on an 8×8 image. Although these two basic operations are very simple and cheap to be implemented they provide an extremely powerful toolbox for image processing applications.

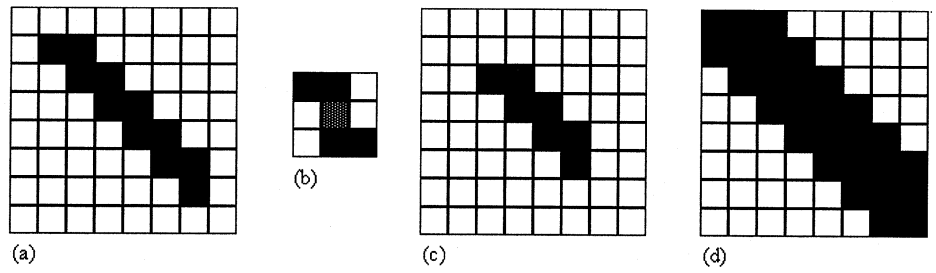

Fig. 3. Erosion (c) and dilation (d) of the image (a) with the structuring element (b).

Thus, the identification of a white pixel is the erosion with a structural element that consists of only one dot at its origin. Whenever the HT algorithms finds a white pixel, it increments the counter for every straight line containing this pixel. By using a structural element which represents a straight line by more than one dot we can significantly reduce the number of counter increments. In particular, all those cases where the white pixel was isolated or consists of only a small white segment do not lead to an increment of any counter. This suppresses the artefacts in the m-c-space. Ideally, a straight line can be represented by two pixels (because there is exactly one straight line intersecting with these two pixels). By using the morphological approach with a structural element consisting of two pixels for every

```
for i := 0 to N − 1 do
    for j := 0 to N − 1 do
        if I(i, j) = 1 then
            for k := 0 to M − 1 do
                if I(i+q_k, j+p_k) = 1 then increment B[m_k, round(j−i·p_k/q_k)]
```

Fig. 4. MHT

```
for k := 0 to N − 1 do
    for i := 0 to N − 1 do
        for j := 0 to N − 1 do
            if I(i, j) = 1 and I(i + q_k, j + p_k) = 1
                then increment B[m_k, round(j − i · p_k/q_k)]
```

Fig. 5. MHT with swapped loops

possible slope we increase a counter if and only if both pixels match in the original image.

Fig. 4 shows an implementation of this method (with $I(i,j)$, $B[m_k, c_l]$ like in Fig. 2 using the structuring elements $B_k = \{(0,0), (q_k, p_k)\}$ for each slope k. In order to produce a parallel implementation on the ISA a much more efficient version of the morphological approach becomes possible if the loops in this program are swapped as in Fig. 5. Now we perform the accumulation operations for the complete image slope by slope. By shifting the image by $(-q_k, -p_k)$ for the slope k we get two copies of the image on top of each other. Then the complete erosion can be performed by one AND-operation of the original image with the shifted image.

In reality, there may occur uncertainties due to rounding effects: Not every straight line has a slope which can be exactly represented by two pixels with a constant distance. The solution of [1] selects a collection of structural elements that are almost equidistant. Skewing the image and choosing an appropriate structural element can avoid this problem. The details of this solution as well as the parallel implementation of the morphological erosion with the shifted image are given in Section 6.

5 SYSTOLA 1024

The parallel computer Systola 1024 is a low cost add on board for standard PCs [12]. The ISA on the board is a 4×4 array of processor chips. Each chip contains 64 processors, arranged as an 8×8 square. This provides 1024 processors as a 32×32

square on the board. In order to exploit the computation capabilities of this unit, it is necessary to provide data and control information at an extremely high speed. Therefore, a cascaded memory concept is implemented on board that forms a fast input and output environment for the parallel processing unit.

For the fast data exchange with the processor array there are rows of intelligent memory units at the northern and western borders of the array. These units are called interface processors. Eight interface processors are integrated on one chip. Each interface processor is connected to its adjacent array processor by a single wire for data transfer in each direction. The interface processors have access to an onboard memory by means of special fast data channels, those at the northern interface chips with the northern board RAM, and those of the western chips with the western board RAM. The northern and the western board RAM can communicate bidirectionally with the PC memory. The data transfer between every two memory units within this hierarchy is controlled by an onboard controller chip. In particular, it controls the channels between the interface processors and the board RAMs as well as between the board RAMs and the bus of the PC (Fig. 6). The controller receives its instructions either directly from the PC as board instructions, or it can operate autonomously. In the second case it receives controller instructions from an instruction queue, which is located on the board, too. The instruction queue is loaded before the execution of an application. The ISA gets its instructions from the ISA program memory. It is also loaded before the execution with the desired application programs. The execution of such programs is started by the controller, as indicated in Fig. 7.

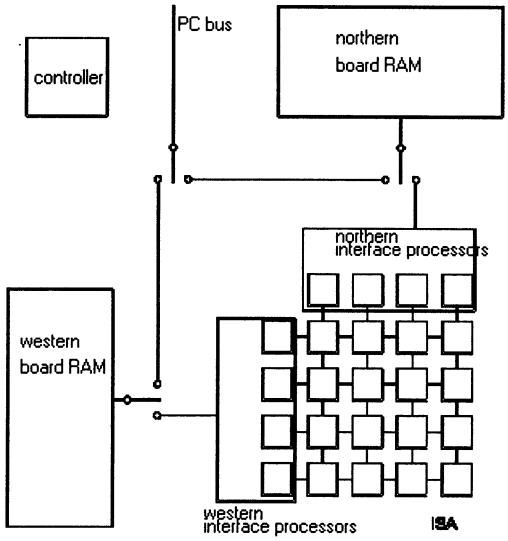

Fig. 6. Data paths in the parallel computer

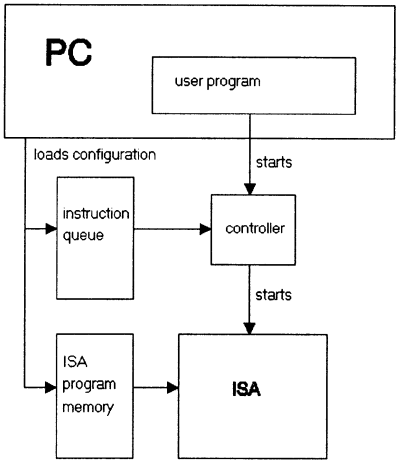

Fig. 7. Execution of a program

At a clock frequency of $f = 50$ MHz and using a word format of $m = 16$ bits each processor can execute $\frac{f}{m} = \frac{50}{16} \cdot 10^6 = 3.125 \cdot 10^6$ word operations per second. The whole array with its 1024 processors performs up to 3200 MIPS (millions of instructions per second). This value characterises the theoretical peak performance. In practice the performance lies, dependent on the application, between 1000 and 3000 MIPS.

One advantage of the programming environment of Systola 1024 is the language *LAISA* which allows to write programs for the ISA in a pascal like notation. A sequential program is written as for one single processor. Due to the systolic instruction execution the same program is executed automatically in every processor where the execution is skewed with respect to time.

6 PARALLEL IMPLEMENTATION OF MORPHOLOGICAL HOUGH TRANSFORM

We have implemented the MHT for binary images of size $N \times N$, $N = 512$. To reduce data transfer and communication time it is useful to store the complete input in the processor array. Thus, the image is partitioned as follows: every processor of Systola 1024 stores a 16×16 subimage in the 16 internal registers $R[1], \ldots, R[16]$. Each of this registers stores 16 adjacent pixels of the same row. We will use the capabilities of the ISA to compute row (column) sums and ringshifts in a very efficient way (see Section 2) for the parallel implementation:

The easiest case is the detection of horizontal (vertical) lines. We use an erosion with a structuring element of two pixels with horizontal (vertical) distance d. Ini-

tially, we shift a copy of the image relative to the original image by d pixels. Now a counter has to be incremented if the pixel of both, the original and the shifted image are set. By means of shearing the image all other line angles can be transformed in horizontal (vertical) position, such that the efficient horizontal (vertical) operations are always applied.

Horizontal Accumulation. This operation produces the sum of the white pixels within each image row. Each processor first counts the white pixels in the corresponding register. This sum is written into the C-register. Afterwards the row sum operation (explained in Section 2) computes the number of white pixels in the whole image row. This operation is repeated 16 times for every subimage row of each processor. The results are collected in the last processor in each row. Because of the limited memory (32 registers) of each processor the results are shifted one processor column to the west in every second iteration step. The ith processor column always collects accumulator entries for the line intercept i. When all processor columns are covered the accumulator array results are transferred to the board RAM. This data transfer can be done concurrently to the computation of the next line slopes. The horizontal accumulation requires 321 processor instructions.

Horizontal Erosion. Let the structuring element have the distance d from the image origin. Then an image row ringshift for d positions and an AND-operation computes the horizontal erosion. In Systola 1024 an image subrow is ringshifted one processor. Afterwards a shifting for d mod 16 positions within each processor is performed. Then a ringshift for a d div 16 processor distance computes the image row ringshift. Finally one AND-operation between the original subrow register and the ringshifted subrow register in each processor calculates the erosion. Afterwards the result is horizontally accumulated (see above). The horizontal erosion requires $96 + 32 \cdot (d \text{ div } 16)$ processor instructions.

Shearing. Shearing is an operation frequently used in scan line image processing [17]. The idea is to reshape the image such that the actual scan line of arbitrary slope appears to be horizontal. This is achieved by shifting the columns with increasing shift distances. Fig. 8 depicts the effect of shearing: A straight line of 45° slope appears to be horizontal if the image is sheared by 45°.

For a line slope $m_k, -1 \leq m_k \leq 1$, and an intercept $c_l, -N < c_l < N$, the accumulation has to be done along the pixels $(i, \text{round}(i \cdot m_k + c_l))$ for all columns $i = 0, \ldots, N-1$. Taking only integer values as intercept parameters the pixels to be accumulated are $(i, \text{round}(i \cdot m_k) + c_l)$. These pixels can be transformed into a horizontal position by ringshifting each column $i = 0, \ldots, N-1$ of the input image by $\text{round}(i \cdot m_k)$ positions. Now the efficient horizontal erosion and accumulation can be applied again.

For an efficient implementation of shearing on Systola 1024 ringshifting along large distances has to be avoided. Thus, the different line slopes are handled in increasing order and the ringshifting distance only needs to incorporate the relative shear between neighbouring angles $\text{round}(i \cdot m_k) - \text{round}(i \cdot m_{k-1})$, for all columns

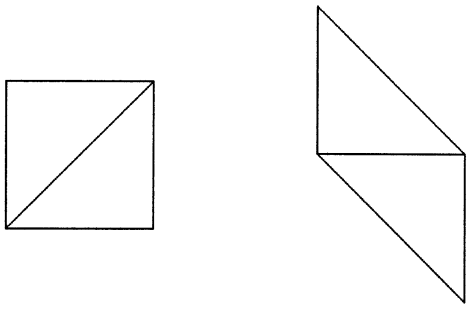

Fig. 8. The right image depicts the left image sheared by 45°

$i = 0, \ldots, N-1$. The distances for relative shearing are known in advance and can be precalculated offline. We assume that the maximal absolute ringhifting distance between two neighbouring angles is $D = 1$. Then, the precalculated shearing distances turn into a binary image row mask: Only the image columns with an entry in the corresponding mask positions have to be ringshifted. The binary mask can be stored in the same way as an image row using only one register per processor column. After the broadcasting of this register along the columns each processor stores the relevant part of the mask. The upper image subrow within each processor ($R1$) is ringshifted in vertical direction (see Section 2) and stored in $R17$. Then each processor can compute the relative shearing by replacing the pixels at seeded mask positions with the pixels of the underlying row:

$$\textbf{for } i := 1 \textbf{ to } 16 \textbf{ do } R[i] := (R[i] \wedge \neg mask) \vee (R[i+1] \wedge mask).$$

Larger absolute ringshift distances between two neighbouring angles of $D > 1$ can be accomplished by D iterations of the above binary case. The shearing for one slope requires $54 \cdot D$ processor instructions.

Because column ringshifting on the ISA is only efficient from southern in northern direction, the above implementation can only be used for line slopes between 0° and −45°. Slopes between 0° and 45° are computed by loading the input image with transposed rows on the ISA and applying the same program. The angles between 45° and 90° (−45° and −90°) are calculated by storing the 16×16 subimage column-wise in each processor and applying the reflected ISA program (swapping row and column selectors, changing west and north, east and south) for 0° to 45° (0° and −45°).

7 PERFORMANCE EVALUATION

To compare the performance of the parallel algorithm in Section 6, we compare the implementation on Systola 1024 with two sequential versions on a PC (Pentium

Processor, 200 MHz, hand optimised assembler code). The first version is the standard version of the HT algorithm (referred to as *P200 standard* in Table 1). The second version is a sequential simulation of the algorithm implemented on Systola 1024 (referred to as *P200 parallel* in Table 1).

Table 1. Comparisons of the average performance of HT/MHT for a 512×512 image and 2048 different slopes on Systola 1024 (including data transfers) and Pentium 200

	Systola 1024	P200 standard	P200 parallel	speedup
HT	0.25 s	7.0 s	5.0 s	28/20
MHT	0.31 s	5.7 s	6.2 s	18/20

For the implementations we use binary images of size $N \times N, N = 512$, and a 16 bit accumulator array consisting of $4N$ slopes and N intercepts. Systola needs 0.25 s (375 processor instructions per slope) for the standard HT and 0.31 s (471 instructions per slope) for the MHT ($D = 1, d = 10$). Since computing time dominates communication time in this application, data transfers can be almost totally hidden as it can be executed concurrently to the computation.

The standard HT (Fig. 2) depends on the number of white pixels in the input image, but has the disadvantage of a complex computation of the accumulator bins to be incremented. This means a computing time of 70.0 s in the worst case (all pixels white). The used edge images from the pick and place process of multichip modules (Fig. 10) have 10% seeded pixels on an average, which corresponds to a computing time of 7.0 s.

The standard MHT (Fig. 4) depends on the number of seeded pixels in the input image and in each eroded image. An average multichip module edge image (Fig. 13) has 10% seeded pixels in the input input and 1% in each eroded image, which leads to a computing time of 5.7 s in the average case. The sequential version of the parallel algorithm in Section 6 is independent of the number of white pixels, but has the advantage of an easier accumulation. This leads to a constant computing time of 5.0 s for the HT and 6.2 s for the MHT for each image.

In order to evaluate the qualitative performance of the MHT we have compared it with the HT on several different images. Fig. 9 shows three lines close together. The relevant part of the accumulator array from the HT and MHT (with distance $d = 50$ for the structuring element) is displayed as a height field in Figs. 10–11. It can be seen that both, HT and MHT produce artefacts (in the figures they are behind the three peaks representing the lines). In case of MHT these artefacts are significantly reduced. The MHT also creates an enhanced accumulator contrast, which simplifies the higher order image analysis.

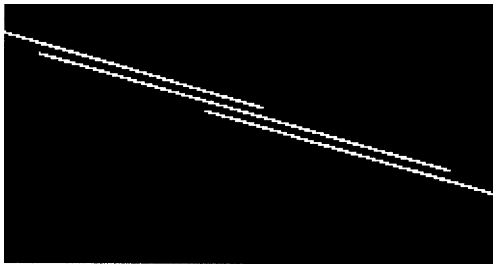

Fig. 9. Original image

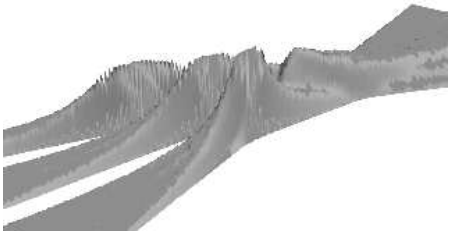

Fig. 10. Accumulator from a HT

Fig. 11. Accumulator from a MHT

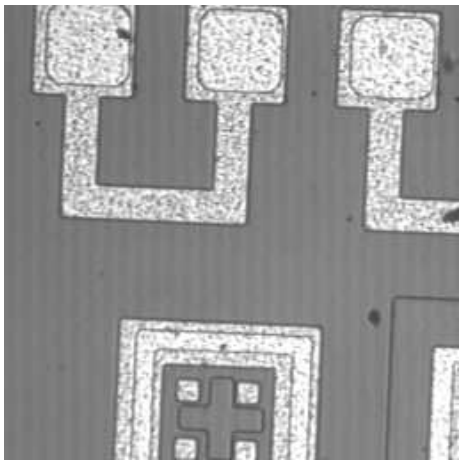

Fig. 12. Original image

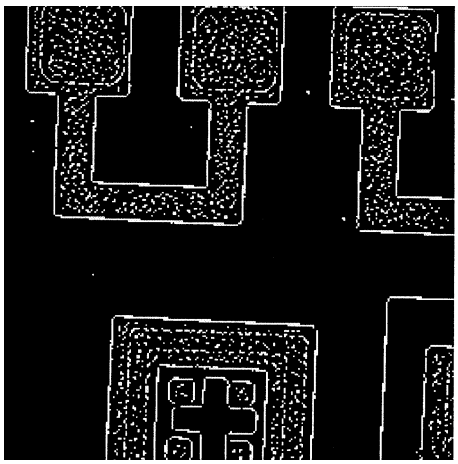

Fig. 13. Edge image

Comparisons of the effectiveness of the HT and MHT methods are displayed in Figs. 12–15. Figs. 12 shows an image from the pick and place process of multichip modules. Fig. 13 shows the result after an edge detector is performed on the image. Figs. 14–15 show the result after a HT and MHT ($d = 10$), where accumulator entries greater than 90 and 80 are detected. The resulting lines are displayed after an AND-operation with the edge image. It can be readily seen that the HT picks up many more false lines than the MHT in the dotted areas.

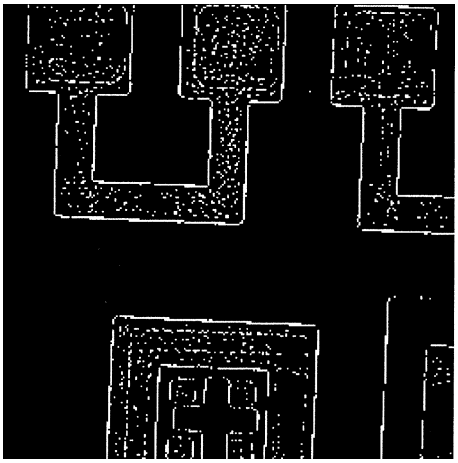

Fig. 14. Result form a HT

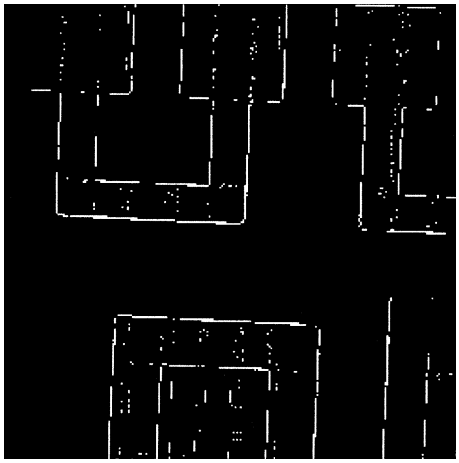

Fig. 15. Result from a MHT

Obviously there is still lots of scope for fine tuning the combination of HT and mathematical morphology to best fit a given application. For example, the structuring elements could be optimised to search for lines of a given thickness (using larger structuring elements) or of given length . It is also possible to tailor the search for dotted or dashed lines.

For the comparison of our approach with other HT algorithms we use as cost function the product of area (A) and time (T). The area is measured simply in the number of processors of the mesh. In order to simplify the comparison of the different implementations of HT, we assume the number of slopes is proportional to the image width N and an ISA of size $O(N^2)$. Because accumulation, shearing, and shifting of the result only takes $O(1)$ time for each slope, it is easy to see that our HT implementation on the $N \times N$ ISA requires $O(N)$ steps, resulting in an asymptotic cost of $AT = O(N^3)$. Futhermore, if we assume the $d \ll N$ this also holds for the presented MHT implementation.

8 COMPARATIVE ALGORITHM ANALYSIS AND CONCLUSIONS

In this paper we have modified a classical algorithm in order to improve its performance, while fitting the characteristics of the flexible ISA architecture. The new algorithm for line detection provides a morphological approach to HT. The algorithm is designed to take advantage of the ability of ISAs to implement accumulation and ringshift operations extremely efficient. The processing time per image shows that the principle of the ISA leads to solutions which are superior with respect to performance and hardware cost. Since image processing is one of the most computation intensive fields in computer science, these results might lead research into the direction of low cost solutions for automatic visual quality control.

The fastest known algorithm for HT in the literature is the one of [8] running in constant time on a reconfigurable architecture. But since it requires $O(N^4)$ processors its AT-complexity is $O(N^4)$ as well. Another approach on the reconfigurable mesh [3] reaches the complexity of $O(N^3 \log N)$. In [14] a systolic array has been designed with a simple processor structure leading to an AT-complexity of $O(N^4 \log N)$. The algorithms of [15] and [16] run on a hypercube and require an AT product of $O(N^3)$ and $O(N^3 \log N)$, respectively. The first HT solution on a mesh that uses techniques from the mathematical morphology has been presented in [1]. Although it is simple and elegant it requires $O(N^2)$ processes and time $O(N^2)$ leading to an AT-complexity of $O(N^4)$. In [18] an $O(N^3)$ algortihm is described on a mesh with a more powerful architecture. The best previously known algorithms on SIMD mesh are those of [7] and [5] reaching the same AT-complexity as our algorithm: $O(N^3)$. In comparison to these our ISA algorithm appears more suitable for practical applications since it requires only very simple processing elements. Furthermore, in terms of measured execution time we expect that the implementation presented here will outperform the implementations of [5] and [7] by an order of magnitude. As an evidence for this claim one may take our implementation on Systola 1024.

The presented ideas for the ISA implementation of the MHT can be extended to efficient ISA implementations of other algorithms in image processing, like the filtered backprojection of parallel and fan beams in computerised tomography.

The current ISA prototype Systola 1024 is a machine based on technology far from being state-of-the-art. Extrapolating to technology used for processors such as the Pentium 200 MHz would lead to a speedup of the ISA by a factor of at least 100. Thus, resulting in processing speeds under 50 ms for the MHT of 512×512 images with 2048 slopes. Such performance can be expected from the already announced next generation Systola board (Systola 4096).

REFERENCES

[1] Beresford-Smith B.—Pham, B.—Schröder, H.: A Parallel Morphological Implementation of the Hough transform. In: Proceedings of 25th HICSS, 1992, pp. 111–119.

[2] Chuang, H. Y. H—Li, C. C.: A Systolic Array Processor for Straight Line Detection by Modified Hough Transform. In: Proc. IEEE Workshop on CAPAIDM, New York, Nov. 1995, pp. 300–304.

[3] Chung, K.-L.—Lin, H.-Y.: Hough Transform on Reconfigurable Meshes. Computer Vision and Image Understanding, Vol. 61, No. 2, 1995, pp. 278–284.

[4] Ferretti, M.—Albanesi, M. G.: Architectures for the Hough Transform: A Survey. In: Proceedings of the IAPR workshop on MVA, Tokyo 1996, pp. 542–551.

[5] Guerra, C.—Hambrusch, S.: Parallel Algorithms for Line Detection on a Mesh. Journal of Parallel and Distributed Computing, Vol. 6, 1989, pp. 1–19.

[6] Illingworth, J.—Kittler, J.: A Survey of the Hough Transform. Computer Vision, Graphics and Image Processing, Vol. 44, 1988, pp. 87–116.

[7] Kannan, C. S.—Chuang, H. Y. H.: Fast Hough transform on a mesh connected processor array. Information Processing Letters, Vol. 33, 1990, pp. 243–248.

[8] Kao, T.-W.—Horng, S.-J.—Wang, Y.-L.–Chung, K.-L.: A Constant Time Algorithm for Computing Hough Transform. Pattern Recognition, Vol. 26, No. 2, 1993, 277–286.

[9] Kunde, M., et al.: The Instruction Systolic Array and its Relation to Other Models of Parallel Computers. Parallel Computing, Vol. 7, 1988, pp. 25–39.

[10] Lang, H.-W.: The Instruction Systolic Array, a parallel architecture for VLSI. Integration, the VLSI Journal, Vol. 4, 1986, pp. 65–74.

[11] Lang, H.-W.: ISA and SISA, Two Variants of a General Purpose Systolic Array Architecture. In: Proceedings of the 2nd Int. Conf. on Supercomputing, 1988, pp. 460–465.

[12] Lang, H.-W.—Maass, R.—Schimmler, M.: Implementation of a 1024-Processor Array Computer as an Add-On Board for Personal Computers. In: Proceedings of HPCN'94, Lecture Notes in Computer Science, Vol. 797, Springer 1994, pp. 487–488.

[13] Leavers, V. F.: Which Hough Transform. Computer Vision, Graphics and Image Processing, Vol. 58, 1993, No. 2, pp. 250–265.

[14] Li, H. F.—Pao, D.—Jakakumar, R.: Improvements and systolic Implementation of the Hough Transform for Straight Line Detection. Pattern Recognition, Vol. 22, No. 6, 1989, pp. 679–706.

[15] Pan, Y.—Chuang, H. Y. H.: Parallel Hough Tranforms Algorithms on SIMD Hypercube Arrays. In: Proc. of Int. Conf. on Parallel Processing, Vol. III, 1990, pp. 83–86..

[16] Ranka, S.—Sahni, S.: Computing Hough transforms on hypercube multicomputers. Journal of Supercomputing, Vol. 4, no. 2, 1990, pp. 169–190.

[17] Robertson, P. K.: Fast perspective views of images using one-dimensional operations. Computer Graphics and Algorithms, Vol. 7, 1987, pp. 47–56.

[18] Rosenfeld, A.—Ornelas, J.—Hung, Y.: Hough Transform Algorithms for Mesh-Connected SIMD Parallel Processors. Computer Vision, Graphics, and Image Processing, Vol. 41, 1988, pp. 293–305.

[19] Schimmler, M.—Lang, H.-W.: The Instruction Systolic Array in Image Processing Applications. In: Proceedings of Europto'96, SPIE Vol. 2784 , 1996, pp. 136–144.

[20] SCHRÖDER, H.: Instruction systolic array – tradeoff between flexibility and speed. Computer Systems, Science and Engineering, Vol. 3, 1988, pp. 83–90.

Manuscript received 19 January 1998; revised 18 June 1999

Bertil SCHMIDT received the diploma degree (M. S.) from the Kiel University, Germany, in 1995 and the Ph.D. degree from Loughborough University, UK, in 1999. He has worked with the companies ISATEC and Siemens in the area of embedded parallel systems. After temporary appointments as Research Associate in Aachen and Karlsruhe he is now a Research Fellow with the Technical University of Braunschweig, Germany. His research interests include computer vision and unconventional parallel architectures.

Manfred SCHIMMLER received the diploma degree and Ph.D. degree from Kiel University, Germany, in 1980 and 1991, respectively. In 1989 he founded the company ISATEC producing instruction systolic arrays for industrial applications. Since 1996 he is Professor for computer engineering first in Stralsund, Germany, and since 1997 with the Technical University of Braunschweig, Germany. His research interests include fine-grained parallel architectures, computer arithmetic, and parallel algorithms.

Heiko SCHRÖDER studied mathematics, physics and computer science at the University of Kiel (Germany), where he received his Ph.D. in computer science in 1977. He was Assistant Professor of computer science at Kansas University (USA), Senior Research Fellow in Canberra (Australia), Professor of microelectronics in Newcastle (Australia) and Professor of CS and HoD in Loughborough (UK). He is currently Professor of CS at NTU in Singapore. His main research interests include massively parallel architectures and algorithms for problems related to sorting, image processing, visualisation, optimisation and fault tolerance.